LeetCode HOT 100

PART 5

Arrays

普通数组

13. 最大子数组和 (No.53) MEDIUM

给你一个整数数组 `nums` ，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组是数组中的一个连续部分。

示例 1：

> **输入：** nums = [-2,1,-3,4,-1,2,1,-5,4]
> **输出：** 6
> **解释：** 连续子数组 [4,-1,2,1] 的和最大，为 6 。

示例 2：

> **输入：** nums = [1]
> **输出：** 1

示例 3：

> **输入：** nums = [5,4,-1,7,8]
> **输出：** 23

提示：

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

进阶： 如果你已经实现复杂度为 `O(n)` 的解法，尝试使用更为精妙的 **分治法** 求解。

13. 最大子数组和 (No.53) MEDIUM

Solution 1 – 前缀和

思路

子数组和变成前缀和以后，之后思想就类似[77.买卖股票的最佳时机]，记录历史最低点然后找最大差值。

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        summ, low, res = 0, inf, -inf
        for num in nums:
            low = min(low, summ)
            summ += num
            res = max(res, summ - low)
        return res
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

Solution 2 – 动态规划

思路

f[i]表示以第i个数字结尾的最大子数组之和，f[i]=max(f[i-1],nums[i])，再用滚动数组降低空间复杂度。

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        summ, res = 0, -inf
        for num in nums:
            summ = max(summ + num, num)
            res = max(res, summ)
        return res
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

13. 最大子数组和 (No.53) MEDIUM

Solution 3 – 分治

思路

原问题是mSum，不断把数组二分，可以用lSum,rSum,mSum,iSum解决。

官方代码可读性很差，这里写一个可读性较好的版本。

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def lSum(l, r):  # [l,r]内以l为左端点的最大子段和
            m = (l + r) // 2
            return nums[l] if l == r else max(lSum(l, m), iSum(l, m) + lSum(m+1, r))
        def rSum(l, r):  # [l,r]内以r为右端点的最大子段和
            m = (l + r) // 2
            return nums[l] if l == r else max(rSum(l, m) + iSum(m+1, r), rSum(m+1, r))
        def mSum(l, r):  # [l,r]内的最大子段和
            m = (l + r) // 2
            return nums[l] if l == r else max(mSum(l, m), mSum(m+1, r), rSum(l, m) + lSum(m+1, r))
        def iSum(l, r):  # [l,r]的区间和
            m = (l + r) // 2
            return nums[l] if l == r else iSum(l, m) + iSum(m + 1, r)

        return mSum(0, len(nums) - 1)
```

时间复杂度：$\mathcal{O}(\log n)$；**空间复杂度**：$\mathcal{O}(\log n)$。

14. 合并区间 (No.56) MEDIUM

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [start_i, end_i]` 。请你合并所有重叠的区间，并返回 *一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间* 。

示例 1：

> **输入：** intervals = [[1,3],[2,6],[8,10],[15,18]]
> **输出：** [[1,6],[8,10],[15,18]]
> **解释：** 区间 [1,3] 和 [2,6] 重叠，将它们合并为 [1,6].

示例 2：

> **输入：** intervals = [[1,4],[4,5]]
> **输出：** [[1,5]]
> **解释：** 区间 [1,4] 和 [4,5] 可被视为重叠区间。

示例 3：

> **输入：** intervals = [[4,7],[1,4]]
> **输出：** [[1,7]]
> **解释：** 区间 [1,4] 和 [4,7] 可被视为重叠区间。

提示：

- $1 <= intervals.length <= 10^4$
- $intervals[i].length == 2$
- $0 <= start_i <= end_i <= 10^4$

14. 合并区间 (No.56) MEDIUM

思路

区间按左端点排序，如果最新得到的区间右端点大于正在遍历区间的左端点就合并，否则直接加入结果。合并时右端点应取最新区间右端点和正在遍历区间右端点的较大值。

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        res = []
        for itv in intervals:
            # 非第一个区间且区间重叠时发生合并
            if res and res[-1][1] >= itv[0]:
                res[-1] = [res[-1][0], max(res[-1][1], itv[1])]
            else:
                res.append(itv)
        return res
```

时间复杂度：$\mathcal{O}(n\log n)$；**空间复杂度**：$\mathcal{O}(\log n)$。复杂度都来源于原地排序。

15. 轮转数组 (No.189) MEDIUM

给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例 1:

> **输入：** `nums = [1,2,3,4,5,6,7], k = 3`
> **输出：** `[5,6,7,1,2,3,4]`
> **解释：**
> 向右轮转 1 步： `[7,1,2,3,4,5,6]`
> 向右轮转 2 步： `[6,7,1,2,3,4,5]`
> 向右轮转 3 步： `[5,6,7,1,2,3,4]`

示例 2:

> **输入：** `nums = [-1,-100,3,99], k = 2`
> **输出：** `[3,99,-1,-100]`
> **解释：**
> 向右轮转 1 步： `[99,-1,-100,3]`
> 向右轮转 2 步： `[3,99,-1,-100]`

提示：

- $1 <= nums.length <= 10^5$
- $-2^{31} <= nums[i] <= 2^{31} - 1$
- $0 <= k <= 10^5$

进阶：

- 尽可能想出更多的解决方案，至少有 **三种** 不同的方法可以解决这个问题。
- 你可以使用空间复杂度为 `O(1)` 的 **原地** 算法解决这个问题吗？

15. 轮转数组 (No.189) MEDIUM

Solution 1 – 数组逆置

思路

使用额外数组的方法就不写在这里了，转换关系为new_nums[(i+k)%n]=nums[i]。

找到轮转后新数组第一个数的原下标start，做三次逆置：nums[0:start],nums[start:n],nums[0:n]。

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1
                right -= 1

        n = len(nums)
        start = (n - k + 1) % n - 1  # 新数组的第一个数原下标
        reverse(0, start-1)
        reverse(start, n-1)
        reverse(0, n-1)
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

15. 轮转数组 (No.189) MEDIUM

Solution 2 – 环状替换

思路

轮转数组可以看成若干个“内部间距为k的子数组”均进行“步长为1”（实质步长为k）的轮转。步长为1的轮转自然容易用空间复杂度为$\mathcal{O}(1)$的算法解决了（只需要一个临时变量），这样的数组有gcd(n,k)个。因此从1到gcd(n,k)就是每个子数组的起始位置start，每个子数组都进行步长为k的轮转，游标i回到start结束。

```python
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        def gcd(a, b):  # 辗转相除法
            return a if b == 0 else gcd(b, a % b)

        n = len(nums)
        for start in range(gcd(n, k)):
            i = start
            while True:
                # 即将把nums[i]移动到nums[(i+k)%n]
                i = (i + k) % n
                # start处存储原先的nums[(i+k)%n]
                nums[start], nums[i] = nums[i], nums[start]
                if i == start:
                    break
```

> 命题：轮转子数组有 `gcd(n,k)` 个（`gcd:最小公约数，lcm:最小公倍数`）
>
> 证明：一只兔子一次跳 `n` 步，一只青蛙一次跳 `k` 步，青蛙和兔子第一次脚印重合的地方在 `lcm(n,k)`（单个子数组遍历了其所有元素），此时青蛙走了 `lcm(n,k)/k` 步（单个子数组的元素数），因此总共有 `n/[lcm(n,k)/k] = nk/lcm(n,k) = gcd(n,k)` 个子数组。

时间复杂度： $\mathcal{O}(n)$

空间复杂度： $\mathcal{O}(1)$

16. 除了自身以外数组的乘积 (No.189) MEDIUM

给你一个整数数组 `nums`，返回 数组 `answer` ，其中 `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积 。

题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。

请 **不要使用除法，** 且在 `O(n)` 时间复杂度内完成此题。

示例 1:

> **输入：** `nums = [1,2,3,4]`
> **输出：** `[24,12,8,6]`

示例 2:

> **输入：** `nums = [-1,1,0,-3,3]`
> **输出：** `[0,0,9,0,0]`

提示：

- `2 <= nums.length <= 10^5`
- `-30 <= nums[i] <= 30`
- 输入 **保证** 数组 `answer[i]` 在 **32 位** 整数范围内

进阶： 你可以在 `O(1)` 的额外空间复杂度内完成这个题目吗？（出于对空间复杂度分析的目的，输出数组 **不被视为** 额外空间。）

16. 除了自身以外数组的乘积 (No.189) MEDIUM

思路

每个数除自身以外的乘积=左前缀积*右前缀积，因此正序、倒序遍历一遍算两次前缀积搞定。注意前缀积初始化为1.

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        res = [1] * n
        for i in range(1, n):
            res[i] = res[i-1] * nums[i-1]  # 左前缀积
        prod = 1  # res占用了，单独一个变量记录右前缀积
        for i in range(n-1, -1, -1):
            res[i] *= prod
            prod *= nums[i]
        return res
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

17. 缺失的第一个正数 (No.41) HARD

给你一个未排序的整数数组 `nums` ，请你找出其中没有出现的最小的正整数。

请你实现时间复杂度为 `O(n)` 并且只使用常数级别额外空间的解决方案。

示例 1：

> **输入：** nums = [1,2,0]
> **输出：** 3
> **解释：** 范围 [1,2] 中的数字都在数组中。

示例 2：

> **输入：** nums = [3,4,-1,1]
> **输出：** 2
> **解释：** 1 在数组中，但 2 没有。

示例 3：

> **输入：** nums = [7,8,9,11,12]
> **输出：** 1
> **解释：** 最小的正数 1 没有出现。

提示：

- $1 <= nums.length <= 10^5$
- $-2^{31} <= nums[i] <= 2^{31} - 1$

17. 缺失的第一个正数 (No.41) HARD

Solution 1 – 原地置换

思路

长度为n的数组缺失的第一个正数只可能是1~n+1，理想情况下数组存储nums[i]=i+1，一旦不符合就返回i+1，如果都符合就返回n+1。想要将原数组变成理想情况，就从第一个元素开始，一旦遇到1到n的数不满足nums[i]≠nums[nums[i]-1]的，直接交换过来让nums[nums[i]-1]位置理想。如果已经理想就不用换；或者当前数不是1到n也不管，因为不关心这些数在什么位置，它如果“挡道”了之后也会被换走。

```python
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        # 理想情况下nums[0]=1,...,nums[n-1]=n,即nums[i]=i+1
        for i in range(n):
            while 1 <= nums[i] <= n and nums[i] != nums[nums[i]-1]:
                x = nums[i]  # 如果nums[nums[i]-1], nums[i]交换会出错
                nums[x-1], nums[i] = x, nums[x-1]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

17. 缺失的第一个正数 (No.41) HARD

Solution 2 – 原地哈希表

思路

如果没有空间复杂度$\mathcal{O}(1)$的限制，可以维护一个哈希表记录1~n是否出现过。但这太浪费哈希表了，因为哈希表value仅存储1/0表示是否出现过，因此先将所有非正数改为n+1后，遍历数组取绝对值，如果1~n出现，就把nums对应位置的数取反表示其存在，最后找第一个正数就知道答案。

```python
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        # 所有非正数改成n+1才不影响后续流程
        nums = [n+1 if num <= 0 else num for num in nums]
        # 之后nums[i]<0表示i+1存在
        for i in range(n):
            # nums[i]可能之前被取反了，由于负数已被清理，可以直接取绝对值看
            x = abs(nums[i])
            if 1 <= x <= n and nums[x-1] > 0:
                nums[x-1] = -nums[x-1]
        for i in range(n):
            if nums[i] > 0:
                return i + 1
        return n + 1
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$